

Tema 8

Herencia y polimorfismo



Índice

- Introducción
- Herencia en C#
- Sobrescritura de métodos
- Constructores en herencia
- Clases y métodos abstractos
- Polimorfismo en C#
- Buenas prácticas en herencia y polimorfismo



- En la programación orientada a objetos (POO), la **herencia** y el **polimorfismo** son conceptos fundamentales que permiten crear sistemas modulares, reutilizables y escalables.
- Ambos conceptos facilitan el diseño de código flexible, donde las relaciones entre clases reflejan conexiones lógicas y funcionales que existen en el mundo real.

1. Introducción



Herencia

- La **herencia** es un mecanismo que permite que una clase (*clase derivada o subclase*) herede miembros (campos, métodos y propiedades) de otra clase (*clase base o superclase*).
- La herencia nos permite establecer una jerarquía de clases que refleje una relación de tipo "**es un**" (por ejemplo, un Perro **es un** Animal).

1. Introducción

- El propósito principal de la herencia es permitir que las clases derivadas puedan aprovechar el comportamiento de las clases base y, en caso necesario, extender o modificar ese comportamiento para sus propias necesidades específicas.

1. Introducción

Polimorfismo

- El **polimorfismo** es la capacidad de utilizar un mismo nombre o interfaz para realizar diferentes operaciones.
- En C#, el polimorfismo permite que un objeto de clase base apunte a un objeto de una clase derivada y ejecute el comportamiento específico de la clase derivada.

1. Introducción

- A lo largo del tema vamos a ver los conceptos de herencia y polimorfismo en profundidad.
- Vamos a ver cómo se implementan en C#, mecanismos que nos van a resultar útiles y también que implicaciones tendrán en nuestras clases.
- Por último veremos una serie de buenas prácticas cuando trabajemos con herencia y polimorfismo en C#.

1. Introducción

- Como hemos visto en este tema y en otros anteriores, la herencia es uno de los 4 pilares de la **POO**.
- La herencia permite que una clase **herede características** (campos, propiedades, métodos) de otra clase.
- Esto resulta útil para **reutilizar código** y crear jerarquías que modelen relaciones entre objetos de forma lógica y estructurada.

2. Herencia en C#

Clases base y clases derivadas

- En C#, la herencia se implementa mediante el uso de clases base y clases derivadas.
- Una **clase base** es una clase que contiene funcionalidad común que se desea reutilizar en otras clases.
- Las **clases derivadas** obtienen de la clase base sus métodos y propiedades, ahorrando la necesidad de repetir código.

2. Herencia en C#

- Una **clase derivada** extiende la funcionalidad de la clase base, permitiendo agregar o modificar los métodos heredados.

Sintaxis básica:

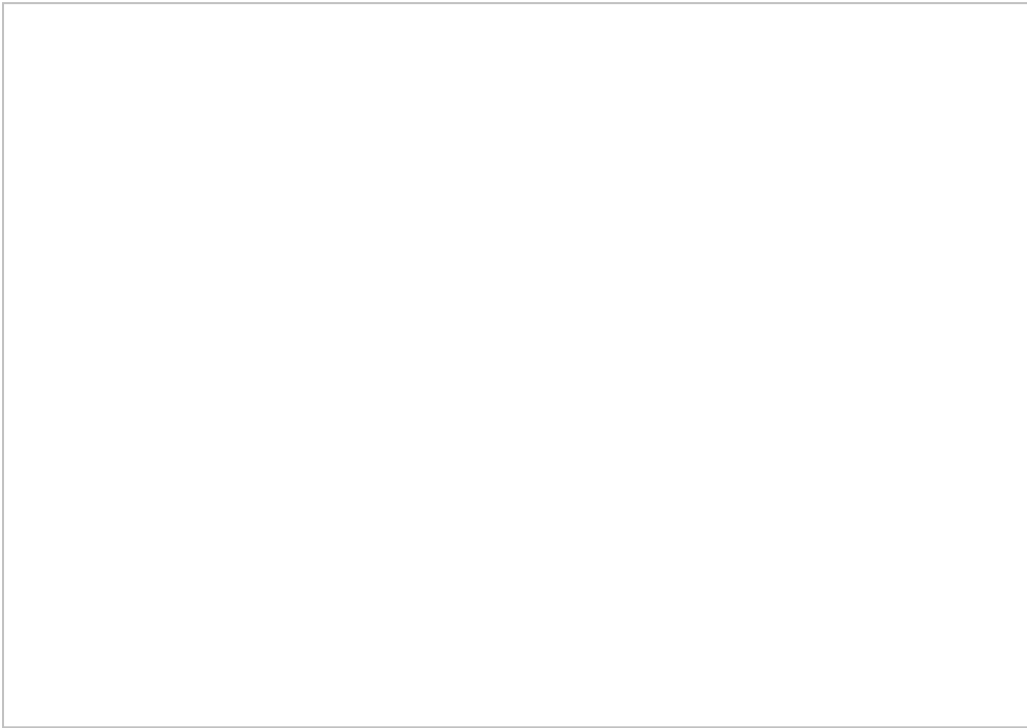
2. Herencia en C#

```
public class ClaseBase
{
    // Miembros de la clase base
}

public class ClaseDerivada : ClaseBase
{
    // Miembros de la clase derivada, que hered
```

Ejemplo

2. Herencia en C#



- Teniendo en cuenta el diagrama anterior, podríamos definir las clases de animal y reptil como:

2. Herencia en C#

```
1 public class Animal
2 {
3     public string Species { get;
4     public Gender Gender { get;
5     public double Weight { get;
6     public int Age { get; set; }
7
8     ...
9
10    public void Eat()
11    {
12        ...
13    }
14
15    public void Move()
16    {
17        ...
18    }
```

```
1 public class Reptile : Animal
2 {
3     public bool IsPoisonous { ge
4
```

```
5      ...  
6  
7      public void ShedSkin()  
8      {  
9          ...  
10     }
```

- En el ejemplo, se define una **clase base** `Animal` que define unas propiedades y comportamientos comunes a todos los animales.
- Después, tenemos diferentes **clases derivadas** que especializan la clase base y además añaden sus propias propiedades o métodos.
- Esto permite estructurar el código en jerarquías lógicas, manteniendo comportamientos comunes en la clase base y añadiendo especificidades en las clases derivadas.

2. Herencia en C#

- Cada clase derivada puede a su vez ser clase base de nuevas clases derivadas, creando una **estructura jerárquica** de varios niveles:

2. Herencia en C#



Modificador protected

- En el tema anterior vimos los modificadores de acceso, entre ellos protected, que establece que un miembro sólo puede ser accedido únicamente por la propia clase o sus derivadas.
- Esto asegura que las clases derivadas puedan usar o modificar estos miembros protegidos, mientras que el acceso directo a estos elementos permanece restringido para el resto del programa.

2. Herencia en C#

- El uso de `protected` en herencia nos proporciona varias ventajas importantes:
- **Control sobre el acceso a datos:** Permite que las clases base controlen cómo se accede y modifica la información sensible o compleja, sin exponer estos datos a clases externas.
- **Reutilización de código:** Las clases derivadas pueden reutilizar métodos o propiedades protegidas, lo que promueve la reutilización de código sin comprometer la encapsulación.

2. Herencia en C#

- El uso de `protected` en herencia nos proporciona varias ventajas importantes:
- **Flexibilidad en el diseño:** `protected` permite que las clases derivadas tengan acceso a la información interna que necesitan para extender o modificar el comportamiento de la clase base, sin tener que recurrir a métodos `public` o exponer el código innecesariamente.

2. Herencia en C#

Ejemplo

2. Herencia en C#

```
1  public class Employee
2  {
3      // Campo protegido accesible solo desde Empleado y
4      protected double Salary;
5
6      ...
7
8      // Método público para obtener el salario
9      public double GetSalary()
10     {
11         return Salary;
12     }
13 }
```

Ejemplo

2. Herencia en C#

```
1  public class FullTimeEmployee : Employee
2  {
3      private double _bonus;
4
5      ...
6
7      public void RaiseSalary(double raise)
8      {
9          Salary += raise; // Modificamos el campo protegido de la
10     }
11
12     public double GetSalaryWithBonus()
13     {
14         return Salary + _bonus; // Accedemos al campo protegido
15     }
```

- En el ejemplo anterior, tenemos una clase base Employee con un campo protected (Salary).
- Desde el exterior, no se puede acceder ni modificar al campo Salary, por lo que creamos un método público para mostrarlo, pero no permitimos que se modifique.

2. Herencia en C#

- La clase derivada `FullTimeEmployee` sí tiene acceso al miembro `Salary`, por lo que puede tanto consultar su valor (método `GetSalaryWithBonus`) como modificarlo (método `RaiseSalary`).
- Los métodos `RaiseSalary` y `GetSalaryWithBonus` permiten personalizar el comportamiento y extender las funcionalidades de la clase base sin exponer el campo `Salary` a clases que no pertenezcan a la jerarquía de `Employee`.

2. Herencia en C#

Limitación de herencia

- En ocasiones, nos encontraremos en una situación en la que no queremos que una clase sea derivada por ninguna otra.
- Por ejemplo si una clase la hemos desarrollado para resolver un problema específico que ya ha quedado resuelto o si por seguridad queremos garantizar que no se modifican las funcionalidades de una clase en concreto.

2. Herencia en C#

- En esos casos, podemos utilizar el modificador **sealed**.
- En C#, la palabra clave **sealed** se utiliza para restringir la herencia de una clase.
- Esto significa que una **clase sellada** no puede ser heredada por otras clases.

2. Herencia en C#

- Sintaxis:
- Ponemos la palabra clave `sealed` delante de `class`. Esto evita que se pueda heredar una clase:
- Si intentamos heredar, se produce un error y no podemos compilar.

2. Herencia en C#

```
1 public sealed class Clas
2 {
3     ...
4 }
```

```
1 public class ClaseDerivada : ClaseBase  
2 {  
3     ...  
}
```

Vídeo práctico

2. Herencia en C#

Vídeo práctico - código Furniture

2. Herencia

```
1  enum WoodType
2  {
3      Pine,
4      Oak,
5      Chestnut,
6      Birch
7  }
8
9  internal class Furniture
10 {
11     protected WoodType WoodType;
12
13     public int Height { get; set; }
14     public int Width { get; set; }
15     public int Depth { get; set; }
16
17     public void Mount()
18     {
19         Console.WriteLine("Montaje del mueble.
```

Vídeo práctico - código Table, Desk

2. Herencia

```
1 internal class Table : Furniture
2 {
3     public virtual int NumberOfLegs { get; :
4     public int SeatCapacity { get; set; }
5 }
6
7 internal class Desk : Table
8 {
9 }
```

Vídeo práctico - código Wardrobe

2. Herencia

```
1  internal sealed class Wardrobe : Furniture
2  {
3      public Wardrobe()
4      {
5          Drawers = new List<string>();
6      }
7
8      public List<string> Drawers { get; set; }
9      public int NumberOfDoors { get; set; }
10
11     public void AddDrawer(string drawer)
12     {
13         Drawers.Add(drawer);
14     }
15 }
```

Vídeo práctico - código Program

2. Herencia

```
1  static void Main(string[] args)
2  {
3      List<Furniture> furnitures = new List<Furniture>()
4      {
5          new Table(){ Height = 60, Width = 90, Depth = 40, Nu
6          new Wardrobe() { Height = 200, Width = 90, Depth = 6
7      };
8
9      foreach (var furniture in furnitures)
10     {
11         furniture.Mount();
12         furniture.ShowInfo();
13         Console.WriteLine();
14     }
```

- En C#, la **sobrescritura de métodos y propiedades** es una técnica fundamental que permite modificar el comportamiento de métodos o propiedades heredados de una clase base en una clase derivada.
- Esta funcionalidad es esencial para la programación orientada a objetos, ya que facilita el uso del **polimorfismo** y permite que una misma operación o método se ejecute de diferentes maneras dependiendo del contexto.

3. Sobrescritura de Métodos

- La sobrescritura se da cuando una clase derivada **redefine** un método o propiedad heredada de la clase base.
- Esto permite que la clase derivada implemente un comportamiento específico, ajustado a sus necesidades, mientras sigue manteniendo la estructura y la interfaz definidas en la clase base

3. Sobrescritura de Métodos

- En C#, para que un método se pueda sobrescribir, este debe estar marcado con la palabra clave **virtual** en la clase base.
- En la clase derivada se utiliza la palabra clave **override** para indicar que se está redefiniendo el método o propiedad virtual de la clase base.

3. Sobrescritura de Métodos

Ejemplo

3. Sobrescritura de Métodos

```
1 public class Animal
2 {
3     public string Species { get; set; }
4     public virtual void Move(int distance)
5     {
6         Console.WriteLine("El animal {0} camina {1} km", Spec
7     }
8 }
9
1 public class Dog : Anim
2 {
3     ...
4 }
```

```
1 public class Bird : Animal
2 {
3     ...
4     public override void Move(int distance)
5     {
6         Console.WriteLine("El pájaro {0} vuela {1} km", Sp
7     }
8 }
```

- En este ejemplo, tenemos una clase base que define un animal que puede moverse. Define un **comportamiento básico** para el método Move en el que el animal camina.
- Este método está marcado como `virtual`, por lo que se puede **sobrescribir**.
- La clase derivada Dog no sobrescribe el método, por lo que se llamará al Move de la clase Animal.
- La clase Bird sí sobrescribe el método, redefiniendo así su comportamiento.

3. Sobrescritura de Métodos

- Nos creamos unos objetos de cada clase para ver su funcionamiento:
- La variable de tipo Dog no sobrescribe el método virtual Move, por lo que ejecuta el método de la clase base.
- La clase Bird sí sobrescribe el método, por lo que ejecuta su propio comportamiento modificado.

3. Sobrescritura de Métodos

```
1 Dog dog = new Dog() { Species = "Per  
2 Bird parrot = new Bird() { Species =  
3  
4 dog.Move(5);
```



- No solo se pueden sobrescribir los métodos, también podemos marcar como `virtual` propiedades para sobrescribirlas en clases derivadas.
- También se puede sellar un método o propiedad sobrescrito para que no se pueda sobrescribir más veces combinando `sealed override` en la declaración.

3. Sobrescritura de Métodos

Acceso a clase base

- En C#, la palabra clave `base` permite a una clase derivada acceder directamente a los miembros de su clase base.
- Esto incluye tanto **métodos** y **propiedades** como **constructores** que hayan sido definidos en la clase base y heredados por la clase derivada.

3. Sobrescritura de Métodos

- La palabra clave base permite que una clase derivada acceda a un método o propiedad de la clase base, incluso si este método ha sido sobrescrito.
- Esto resulta útil cuando el comportamiento original de la clase base es necesario como parte de la lógica en la clase derivada.

3. Sobrescritura de Métodos

Ejemplo

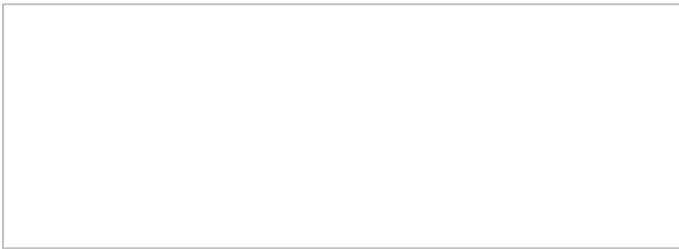
3. Sobrescritura de Métodos

```
1 public class Employee
2 {
3     public string Name { get; set; }
4     public string Company { get; set; }
5
6     ...
7
8     public virtual void PrintInfo()
9     {
10         Console.WriteLine("Empleado: {0}.\nEmpresa: {1}.",
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649

```

- En el ejemplo anterior, la clase derivada `FullTimeEmployee` ejecuta el método `PrintInfo` de la clase base y después añade más funcionalidad.
- El resultado de ejecutar `PrintInfo` en un objeto de la clase derivada es la combinación de ambos métodos, ya que el segundo está llamando al primero:

3. Sobrescritura de Métodos



Diferencias entre sobrecarga y sobrescritura

- En el tema anterior vimos la sobrecarga de métodos, en la que se definen varios métodos con el mismo nombre (pero diferente firma) en la misma clase.
- Tanto la sobrecarga como la sobrescritura permiten el uso de métodos con el mismo nombre, pero se aplican en **contextos diferentes** y no deben confundirse.

3. Sobrescritura de Métodos

- La **sobrecarga** trabaja dentro de la misma clase y depende de la cantidad y tipos de parámetros.
- Se usa cuando necesitas que un método realice la **misma operación general**, pero con diferentes tipos o números de parámetros.
- Esto permite flexibilidad en la cantidad de argumentos que un método puede aceptar.

3. Sobrescritura de Métodos

- La **sobrescritura** se utiliza en clases derivadas para modificar el comportamiento de métodos de la clase base y se basa en la herencia y el polimorfismo.
- Es útil cuando tienes una **jerarquía de herencia** y necesitas que un método se comporte de manera diferente en una clase derivada.
- Esto permite especializar el comportamiento de los métodos en clases derivadas, manteniendo el código base común.

3. Sobrescritura de Métodos

Vídeo práctico

3. Sobrescritura de Métodos

Vídeo práctico - código Furniture

3. Sobrescritura de Métodos

```
1  enum WoodType
2  {
3      Pine,
4      Oak,
5      Chestnut,
6      Birch
7  }
8
9  internal class Furniture
10 {
11     protected WoodType WoodType;
12
13     public int Height { get; set; }
14     public int Width { get; set; }
15     public int Depth { get; set; }
16
17     public virtual void Mount()
18     {
19         Console.WriteLine("Montaje del mueble...");
20     }
21 }
```

Vídeo práctico - código Table

3. Sobrescritura de Métodos

```
1  internal class Table : Furniture
2  {
3      public virtual int NumberOfLegs { get; set; }
4      public int SeatCapacity { get; set; }
5
6      public override void Mount()
7      {
8          Console.WriteLine("Montaje de la mesa...");
9      }
10
11     public override void ShowInfo()
12     {
13         Console.WriteLine("Mesa");
14         base.ShowInfo();
15         Console.WriteLine("Número de patas: {0}. Capacidad
16     }
```

Vídeo práctico - código Wardrobe

3. Sobrescritura de Métodos

```
1  internal sealed class Wardrobe : Furniture
2  {
3      public Wardrobe()
4      {
5          Drawers = new List<string>();
6      }
7
8      public List<string> Drawers { get; set; }
9      public int NumberOfDoors { get; set; }
10
11     public void AddDrawer(string drawer)
12     {
13         Drawers.Add(drawer);
14     }
15
16     public override void Mount()
17     {
18         Console.WriteLine("Montaje del armario...");
19     }
20
21     public override void ShowInfo()
```

Vídeo práctico - código Desk

3. Sobrescritura de Métodos

```
1  internal class Desk : Table
2  {
3      public override int NumberOfLegs
4      {
5          get => base.NumberOfLegs;
6          set
7          {
8              base.NumberOfLegs = value;
9              Console.WriteLine("Patatas del escritorio: " + value);
10         }
11     }
12
13     public override void ShowInfo()
14     {
15         Console.WriteLine("Escritorio");
16         Console.WriteLine("Alto: {0}. Ancho: {1}. Profundo: {2}");
17         Console.WriteLine("Número de patas: {0}. Capacidad: {1}");
18     }
19 }
```

Vídeo práctico - código Program

3. Sobrescritura de Métodos

```
1  static void Main(string[] args)
2  {
3      List<Furniture> furnitures = new List<Furniture>()
4      {
5          new Table(){ Height = 60, Width = 90, Depth = 40, Nui
6          new Desk() { Height = 60, Width = 300, Depth = 60, Ni
7          new Wardrobe() { Height = 200, Width = 90, Depth = 60
8      };
9
10     foreach (var furniture in furnitures)
11     {
12         furniture.Mount();
13         furniture.ShowInfo();
14         Console.WriteLine();
15     }
```

- En C#, los constructores **no se heredan directamente** de una clase base a una clase derivada.
- Sin embargo, los constructores de las clases derivadas **deben invocar** un constructor de la clase base.
- Si existe un constructor por defecto en la clase base (sin parámetros), se llamará automáticamente a ese constructor en los constructores de las clases derivadas.

4. Constructores en herencia

- Si no existe constructor sin parámetros en la clase base o si queremos llamar a un constructor diferente, utilizaremos la palabra clave **base**.
- Si no hay definido un constructor en la clase derivada, también se intenta llamar al constructor sin parámetros de la clase base. En caso de no existir, resultará en un error y no dejará compilar.

4. Constructores en herencia

Ejemplo

4. Constructores en herencia

```
1  public class Employee
2  {
3      public Employee(string name, string company)
4      {
5          Name = name;
6          Company = company;
7      }
8
9      ...
10 }
```

```
1 public class FullTimeEmployee : Employee
2 {
3     public FullTimeEmployee(string name, string company, double bonus)
4     {
5         _bonus = bonus;
6     }
7
8     public FullTimeEmployee(string name)
9     {
10    }
11
12    ...
```

- En el ejemplo anterior, la clase `FullTimeEmployee` tiene 2 constructores.
- El primero de ellos llama al constructor de la clase base con dos de los parámetros que recibe (llamada : `base(name, company)`).
- El segundo constructor no llama a ningún constructor de la clase base. Como no hay definido ningún constructor sin parámetros en `Employee`, daría un **error** y no compilaría.

4. Constructores en herencia

- En la jerarquía de constructores, siempre se ejecuta en **primer lugar** el constructor de la clase base y posteriormente el de la clase derivada.
- Este orden de inicialización asegura que la clase derivada parta de una base completamente inicializada, heredando los estados necesarios de la clase superior.

4. Constructores en herencia

Vídeo práctico

4. Constructores en herencia

Vídeo práctico - código Furniture

4. Constructores en herencia

```
1  internal class Furniture
2  {
3      protected WoodType WoodType;
4
5      public Furniture(int height, int width, int d
6      {
7          Height = height;
8          Width = width;
9          Depth = depth;
10     }
11
12     public Furniture(int height, int width, int d
13     {
14         WoodType = woodType;
15     }
16
17     public int Height { get; set; }
18     public int Width { get; set; }
19     public int Depth { get; set; }
20
```

Vídeo práctico - código Table

4. Constructores en herencia

```
1  internal class Table : Furniture
2  {
3      public virtual int NumberOfLegs { get; set; }
4      public int SeatCapacity { get; set; }
5
6      public Table(int height, int width, int depth
7          : base(height, width, depth)
8      {
9          NumberOfLegs = numberOfLegs;
10         SeatCapacity = seatCapacity;
11     }
12
13     public override void Mount()
14     {
15         Console.WriteLine("Montaje de la mesa...")
16     }
17
18     public override void ShowInfo()
19     {
20         Console.WriteLine("Mesa");
```


Vídeo práctico - código Desk

4. Constructores en herencia

```
1  internal class Desk : Table
2  {
3      public Desk(int height, int width, int depth,
4                  : base(height, width, depth, numberOfLegs
5      {
6
7      }
8
9      public override int NumberOfLegs
10     {
11         get => base.NumberOfLegs;
12         set
13         {
14             base.NumberOfLegs = value;
15             Console.WriteLine("Patatas del escritor
16         }
17     }
18
19     public override void ShowInfo()
20     {
```

Vídeo práctico - código Wardrobe

4. Constructores en herencia

```
1  internal sealed class Wardrobe : Furniture
2  {
3      public Wardrobe(int height, int width, int de
4          : base(height, width, depth, WoodType.Oak
5      {
6          NumberOfDoors = numberOfDoors;
7          Drawers = new List<string>();
8      }
9
10     public List<string> Drawers { get; set; }
11     public int NumberOfDoors { get; set; }
12
13     public void AddDrawer(string drawer)
14     {
15         Drawers.Add(drawer);
16     }
17
18     public override void Mount()
19     {
20         Console.WriteLine("Montaie del armario...
```

Vídeo práctico - código Program

4. Constructores en herencia

```
1  static void Main(string[] args)
2  {
3      List<Furniture> furnitures = new List<Furn
4      {
5          new Table(64, 90, 40, 4, 4),
6          new Desk(60, 300, 60, 6),
7          new Wardrobe(200, 90, 60, 3)
8      };
9
10     foreach (var furniture in furnitures)
11     {
12         furniture.Mount();
13         furniture.ShowInfo();
14         Console.WriteLine();
15     }
```

- Las **clases abstractas** y los **métodos abstractos** en C# son conceptos clave en la programación orientada a objetos que ayudan a definir comportamientos y estructuras que las clases derivadas deben implementar.
- Estos elementos proporcionan una manera estructurada de establecer una base común de características, mientras dejan detalles específicos para las clases que hereden de ellas.

5. Clases y métodos abstractos

Clases abstractas

- Una clase abstracta es una clase que **no puede instanciarse** directamente.
- Se utiliza principalmente como una clase base, que establece ciertos comportamientos y atributos comunes para las clases derivadas.
- Las clases abstractas pueden contener métodos "normales" (con implementación) y métodos abstractos (sin implementación).

5. Clases y métodos abstractos

- Aunque las clases abstractas pueden tener constructores, estos **no se pueden invocar** para crear un objeto, únicamente se pueden utilizar en los constructores de las clases derivadas.
- Conceptualmente, una clase abstracta representa una **entidad genérica** que será implementada por otras específicas.
- Por ejemplo, la clase `Animal` podría ser un buen ejemplo de clase abstracta ya que es un concepto genérico que para existir depende de otros específicos.

5. Clases y métodos abstractos

Métodos abstractos

- Un **método abstracto** es un método declarado dentro de una clase abstracta que no tiene implementación.
- Su propósito es definir una firma que las clases derivadas deben implementar, asegurando que cada subclase tenga su propia versión específica del método.
- Un método abstracto **obliga** a las clases derivadas a proporcionar una implementación específica.

5. Clases y métodos abstractos

Ejemplo

5. Clases y métodos abstractos

```
1 public abstract class Employee
2 {
3     public string Name { get; set; }
4     public abstract double CalculateSalary
5     public void ShowInfo()
6     {
7         Console.WriteLine("Empleado: " +
8     }
9
10 }
```

```
1 public class FullTimeEmployee : Employee
2 {
3     public double YearlySalary { get; set; }
4     public override double CalculateSalary
5     {
6         return YearlySalary / 12;
7     }
8 }
```

```
1 public class HourlyEmployee
2 {
3     public double HourTariff { get; set; }
4     private int _workedHours;
5 }
```



```
6 public override double Calcul  
7 {  
8     return HourTariff * _work  
9 }
```

- En el ejemplo, la clase abstracta `Employee` no se puede instanciar.
- Sin embargo, define el comportamiento de un método (`ShowInfo`).
- También define un método abstracto (`CalculateSalary`), sin implementación, que cada clase derivada debe implementar obligatoriamente.

5. Clases y métodos abstractos

Clase abstracta vs interfaz

- En el tema anterior vimos el concepto de interfaz, que define un contrato que deben cumplir las clases que lo cumplan, pero no implementa ningún comportamiento.
- A pesar de que tanto las clases abstractas como las interfaces permiten definir métodos que deben ser implementados por clases derivadas, existen diferencias clave entre ambos.

5. Clases y métodos abstractos

Clase abstracta vs interfaz

5. Clases y métodos abstractos

	<i>Clase Abstracta</i>	<i>Interfaz</i>
Propósito	Proporciona una base para la herencia	Define un contrato sin implementación
Implementación de métodos	Puede contener métodos con o sin implementación	No hay implementación de métodos
Propiedades	Puede tener campos, propiedades y métodos	Solo define métodos y propiedades (sin campos)
Herencia múltiple	Una clase solo puede heredar de	Una clase puede implementar

	una clase abstracta	múltiples interfaces
Uso	Para relaciones " <i>es un(a)</i> " con base común	Para definir contratos específicos de comportamiento

Vídeo práctico

5. Clases y métodos abstractos

Vídeo práctico - código Furniture

5. Clases y métodos abstractos

```
1  internal abstract class Furniture
2  {
3      protected WoodType WoodType;
4
5      public Furniture(int height, int width, int depth)
6      {
7          Height = height;
8          Width = width;
9          Depth = depth;
10     }
11
12     public Furniture(int height, int width, int depth,
13     {
14         WoodType = woodType;
15     }
16
17     public int Height { get; set; }
18     public int Width { get; set; }
19     public int Depth { get; set; }
20
21     public abstract void Mount();
22 }
```


- El polimorfismo es uno de los 4 pilares fundamentales de la programación orientada a objetos.
- Permite que un método o propiedad de una clase se **comporte de distintas maneras** en función del tipo de objeto que lo llame.
- En C#, el polimorfismo se logra principalmente a través de la herencia y el uso de métodos sobrescritos o a través de las interfaces.

6. Polimorfismo en C#

- **Polimorfismo** significa "muchas formas" y permite que los objetos de diferentes clases se manejen como si fueran de una clase base común.
- Este concepto proporciona una mayor flexibilidad en el código, ya que permite escribir métodos más generales que pueden trabajar con diferentes tipos de objetos sin conocer los detalles específicos de cada uno.

6. Polimorfismo en C#

Beneficios

El uso de polimorfismo en C# nos ofrece una serie de beneficios:

- **Facilita la extensión del código:** Al permitir que las clases derivadas implementen o modifiquen sus propios métodos, podemos agregar fácilmente nuevos tipos de comportamiento sin cambiar el código existente.

6. Polimorfismo en C#

- **Simplifica el código:** El uso de métodos polimórficos reduce la necesidad de usar múltiples `if` o `switch` para verificar el tipo de objeto.
- **Aumenta la reutilización del código:** Al definir un comportamiento común en una clase base, las clases derivadas pueden reutilizarlo y adaptarlo a sus necesidades.

6. Polimorfismo en C#

Ejemplo

- Si tenemos una clase base Transporte y las clases derivadas de ella Coche, Barco y Avión.
- La clase base define un método para conducir un vehículo, pero cada especialización se encarga de cómo realizarlo.
- Si trabajamos a nivel de Transporte, podemos llamar a la función de conducir, independientemente de la clase a la que pertenezca cada objeto.

6. Polimorfismo en C#

Ejemplo

- Llamamos al Move de cada objeto y ni siquiera tenemos que saber el tipo de clase que es ya que sabemos que es un Transport.

6. Polimorfismo en C#

```
1  Transport[] transports = new Transport[]
2  {
3      new Car(),
4      new Ship(),
5      new Plane()
6  };
7
```

```
8  foreach (Transport transport in transports)
9  {
10     transport.Move();
```

Operadores `is` y `as`

- En C#, los operadores `is` y `as` son utilizados para trabajar con conversiones de tipos y comprobación de tipos en tiempo de ejecución.
- Ambos permiten verificar y convertir objetos de manera segura y controlada.
- Vamos a ver cómo funcionan y en qué casos son útiles.

6. Polimorfismo en C#

OPERADOR IS

- El operador `is` se usa para comprobar si un objeto es de un tipo específico.
- Este operador evalúa a `true` si el objeto puede ser tratado como el tipo especificado y a `false` en caso contrario.
- Es particularmente útil para verificar si un objeto es de un tipo o implementa una interfaz antes de realizar alguna operación en él.

6. Polimorfismo en C#

Ejemplo

- Realizamos una comprobación del tipo de clase del objeto. Si el objeto es del tipo especificado, la operación `is` devuelve `true`.

6. Polimorfismo en C#

```
1 if(transport is Ship)
2 {
3     Console.WriteLine("El transporte es ur
```

OPERADOR AS

- El operador as realiza una conversión de tipos segura.
- Si el objeto no es del tipo especificado, devuelve `null` en lugar de lanzar una excepción.
- Este operador es útil cuando se desea convertir un objeto a un tipo específico sin riesgo de error, ya que devuelve `null` si la conversión falla.

6. Polimorfismo en C#

Ejemplo

- Convertimos un objeto a una clase en concreto. Si el objeto no es de esa clase, obtendremos un `null`.
- En caso contrario, obtendremos el objeto de la clase especificada.

6. Polimorfismo en C#

```
1 Ship ship = transport as  
2 if(ship != null)  
3 {  
4     ship.CastAnchor();  
5 }
```

DIFERENCIAS ENTRE IS Y AS

- Usamos **is** cuando necesitamos verificar el tipo del objeto sin realizar la conversión.
- Usamos **as** cuando necesitamos realizar una conversión y evitar excepciones si el objeto no es del tipo correcto.

6. Polimorfismo en C#

Operador	Propósito	Comportamiento en caso de fallo
is	Comprueba si un objeto es de un tipo específico.	Devuelve true o false, no afecta al objeto.

as	Intenta convertir un objeto al tipo especificado.	Devuelve <code>null</code> si falla la conversión.
-----------	---	--

Vídeo práctico

6. Polimorfismo en C#

Vídeo práctico - código Furniture

6. Polimorfismo en C#

```
1  static void Main(string[] args)
2  {
3      List<Furniture> furnitures = new List<Furniture>(
4      {
5          new Table(64, 90, 40, 4, 4),
6          new Desk(60, 300, 60, 6),
7          new Wardrobe(200, 90, 60, 3),
8          new Table(64, 180, 90, 4, 6),
9          new Wardrobe(100, 90, 4, 2)
10     };
11
12     foreach (var furniture in furnitures)
13     {
14         furniture.Mount();
15         furniture.ShowInfo();
16
17         if(furniture is Wardrobe)
18         {
19             ((Wardrobe)furniture).AddDrawer("Calcetir
20         }
21
22         var wardrobe = furniture as Wardrobe;
```

- La herencia y el polimorfismo son principios fundamentales en la programación orientada a objetos.
- Bien aplicados, permiten estructurar el código de manera flexible, reutilizable y fácil de mantener.
- Sin embargo, su uso inadecuado puede llevar a **problemas de complejidad** y rigidez en el diseño.
- A continuación, vamos a ver algunas buenas prácticas que te ayudarán a aplicar estos conceptos de manera efectiva en C#.

7. Buenas prácticas en herencia y polimorfismo

Cuándo usar herencia

- La herencia establece una relación de tipo "*es-un*" entre una clase base y una clase derivada.
- Antes de usarla, tenemos que estar seguros de que la clase derivada realmente representa una versión especializada de la clase base.
- Por ejemplo, un Coche es un tipo de Vehículo, pero un Cliente no debería ser una subclase de Coche simplemente porque ambos tienen atributos como Nombre o Id.

7. Buenas prácticas en herencia y polimorfismo

- La composición (es decir, incluir una clase dentro de otra como un atributo) suele ser más flexible y menos propensa a errores que la herencia.
- Si una clase no representa una versión especializada de otra, es probable que la composición sea una mejor elección.
- Por ejemplo, parece más lógico que un `Coche` tenga un objeto del tipo `Motor` en lugar de ser una subclase de `Motor`.

7. Buenas prácticas en herencia y polimorfismo

Limitación de herencia

- Demasiados niveles de herencia pueden llevar a jerarquías complicadas y difíciles de entender o modificar.
- Además, profundizar demasiado la herencia puede llevar a problemas como la “fragilidad de la base”, donde un cambio en la clase base tiene consecuencias imprevistas en las clases derivadas.
- Es recomendable mantener jerarquías de herencia llanas y evitar los anidamientos innecesarios.

7. Buenas prácticas en herencia y polimorfismo

- También la sobrecarga de métodos puede hacer el código más difícil de leer si se usa en exceso.
- En cuanto a la sobrescritura, es importante seguir un propósito claro, por ejemplo, para especializar o extender un comportamiento específico.
- Los métodos `virtual` y `override` deben usarse cuando se quiere que una clase derivada tenga la libertad de modificar un comportamiento.

7. Buenas prácticas en herencia y polimorfismo

Uso de clases abstractas

- Las clases abstractas son útiles para definir un comportamiento común para un grupo de clases relacionadas, sin implementar todos los detalles.
- Una clase abstracta permite declarar métodos abstractos que las clases derivadas deben implementar.
- Esto facilita la creación de jerarquías de herencia que comparten características comunes pero que pueden especializarse en cada subclase.

7. Buenas prácticas en herencia y polimorfismo

Principio de Sustitución de Liskov

- El Principio de Sustitución de Liskov (**LSP**, por sus siglas en inglés) establece que un objeto de una clase derivada debería poder reemplazar a un objeto de la clase base sin alterar el correcto funcionamiento del programa.
- Es decir, donde sea que un objeto de la clase base sea esperado, debería ser posible utilizar cualquier clase derivada de ella sin problema.
- Deberíamos cumplirlo **siempre** que implementemos herencia de clases.

7. Buenas prácticas en herencia y polimorfismo

Ejemplo

- Si tenemos un método que trabaja con tipos `Transport` y pasamos como parámetro un tipo `Car`, el objeto debe comportarse correctamente como un tipo `Transport`.

7. Buenas prácticas en herencia y polimorfismo

```
1 public void Park(Transport tr
2 {
3     ...
4
5 Car car = new Car();
6 // Utilizamos un objeto Car (d
7 // donde "piden" un Transport
8 // Todo debe funcionar correct
9 Park(car);
```

Diapositiva final

En este tema hemos aprendido acerca de la herencia y el polimorfismo en C#, 2 de los 4 pilares fundamentales de la POO.

Speaker notes